

Project 4

The big idea

This is a synthesis project. We're going to be putting together several things we've learned:

- Hierarchical scene management
- Interactive Cameras
- Loading meshes from files

Consider it a more challenging version of project 3. You may want to reuse your code from that project.

The goal

Your goal is to produce a scene with the following components:

- At the middle of the scene, at the origin, a cube, rotating at a rate of one rotation every 16 seconds.
- Rotating around the cube will be three meshes. One of them must be a teapot, one of them must be a cow, and I'll leave the third up to you. These must be child objects of the cube. You can color these objects however you'd like, as long as the contours are visible (that means no solid colors). Random colors are fine. It's okay if you add additional rotation to these objects for fun.
- There must be a smaller object rotating around the cow. You can pick the mesh and the direction of the orbit.
- The camera must be interactive. You must be able to move it with WASD, and the left and right arrows. (As long as you're not going for the camera bonus objective, you don't need to support looking up and down or roll to avoid needing to re-implement pitch/yaw/roll or axis-angle)
- The camera must have a teapot attached to it, such that the camera is on top of the top of the teapot, and the spout can be seen pointing forward into the screen. This means the teapot should be a child of the camera.

All of this must be done using hierarchical scene management (e.g., a parent-child scene). I recommend looking at [sample11](#) if you need a good base.

Right



Description:

- At the middle is a cube (see `right.mp4` to verify the rotation speed).
- There are three meshes directly rotating around the cube:
 1. A teapot
 2. A cow
 3. I chose the armadillo as my third
 - All are child objects of the cube.
 - All of them are colored such that their contours are visible. Random colors would have been okay, but I computed the vertex normals myself and treated the normals as colors because I thought it would look cool. (If you want to know how to do this, I'm happy to talk about it. We'll end up learning when we get to lighting soon.)
- The cow is being orbited by the bunny. I made it orbit around its x axis.
- The camera is interactive (visible in `right.mp4`)
- The teapot is attached to the camera. Pretend this is a very bizarre weapon in a first-person action game.

Bonus objectives

As always, if you do any of these, leave a Canvas comment on your submission to get credit.

1. If you make the object at the center of the scene a pentagonal prism, your grade for this assignment will replace your grade for project 1, if this assignment's grade is higher.
2. If you make the object at the center be fully textured with **one texture** and having **a different texture for each side**, this assignment's grade will replace your grade for project 2 if it is higher.
3. If you add all the same camera controls that you had for project 3 (q and e to rotate, t to look at one of the objects rotating around the cube, r to reset the camera), this assignment's grade will replace your grade for project 3 if it is higher.

Grading

Hints

- You can get away without implementing pitch/yaw/roll for this one (unless you're attempting the camera bonus objective).
- There is no standard for how big a 3D model should be. Some of them are a little too big (*cough* armadillo) and some are really small (*cough* bunny).
- I just need to be able to see the models' contours. You can use random colors like I did in a sample, but you can also do depth-buffer things, or even texture them in blender. The trick I used in the screenshot was to interpret their normal vector as a color. I computed the normal vectors when loading the OBJ file: compute the normal for each face, then for each vector, add the adjacent face normals together and normalize the resulting vector. That's a simple algorithm that will come in handy when we do lighting.
- The camera does not have to be attached to the root...
- ...but remember that our depth-first algorithm won't find anything that isn't attached to the root. You might need to take special care to draw the teapot attached to the camera.
- Make sure the teapot doesn't clip into the camera. Your perspective matrix determines how close it can get.